SQL 注入漏洞与防护

任务 1：数据库与查询条件基础说明

1. 页面中的输入为什么会与数据库查询有关

因为页面需要动态展示用户关心的数据，而这些数据存在数据库里。
比如“图书查询”页面，用户输入“书名关键词”，页面就要从图书表中查出匹配的书并显示。
没有输入，页面就只能展示固定内容，失去了交互和个性化功能。

2. 一个最小业务链路是什么

用户输入 → 页面接收 → 程序构造查询 → 数据库执行 → 返回结果 → 页面展示
以图书查询为例：
用户在输入框输入：`三体`
页面把 `三体` 传给后端
后端执行类似：

```
sql
SELECT * FROM books WHERE title LIKE '%三体%'
```

数据库返回匹配的图书
页面显示结果
这就是 输入条件 → 数据库查询 → 返回结果。

3. 用户输入是如何进入查询条件的

1. 用户在前端输入框输入内容
2. 前端通过 HTTP 请求（GET/POST）把输入发送给后端
3. 后端代码从请求参数中取出输入值
4. 直接拼接或通过参数化方式放入 SQL 语句中，成为 `WHERE` 条件的一部分

例（伪代码，非安全写法）：

```
python
keyword = request.GET.get("keyword")
sql = "SELECT * FROM books WHERE title LIKE '%" + keyword + "%'"
```

此时用户的 `三体` 就进入了查询条件。

4.为什么这为 SQL 注入创造了可能性

因为用户输入可以被恶意构造，而不是预期的普通关键词。
仍以上面不安全拼接为例，如果用户输入：

```
三体' OR '1'='1
```

那么拼接后的 SQL 变成：

```sql
SELECT * FROM books WHERE title LIKE '%三体' OR '1'='1%'
```

``1'='1'` 永远为真，可能把整张表数据都查出来，甚至执行更危险的语句（如删除表）。

任务 2：DVWA SQL Injection Low 现象记录

1. 实验页面或实验目标

实验页面：DVWA（Damn Vulnerable Web Application）的 SQL Injection 漏洞测试页面，Low 安全级别。
实验目标：通过输入用户 ID 查询用户信息，测试是否存在 SQL 注入漏洞，观察正常输入与恶意输入之间的差异。

2. 普通输入与异常输入的对比

对比项	普通输入	异常输入
输入内容	1	1’or 1=1#
返回结果	仅返回 ID=1 的用户（First name: admin, Surname: admin）	返回数据库中所有用户（admin、Gordon、Hack、Pablo、Bob 共 5 条记录）
结果数量	1 条	多条

截图对比说明：
第一张图：输入 1 → 返回单条用户记录
第二张图：输入 1’or 1=1# → 返回全部用户记录

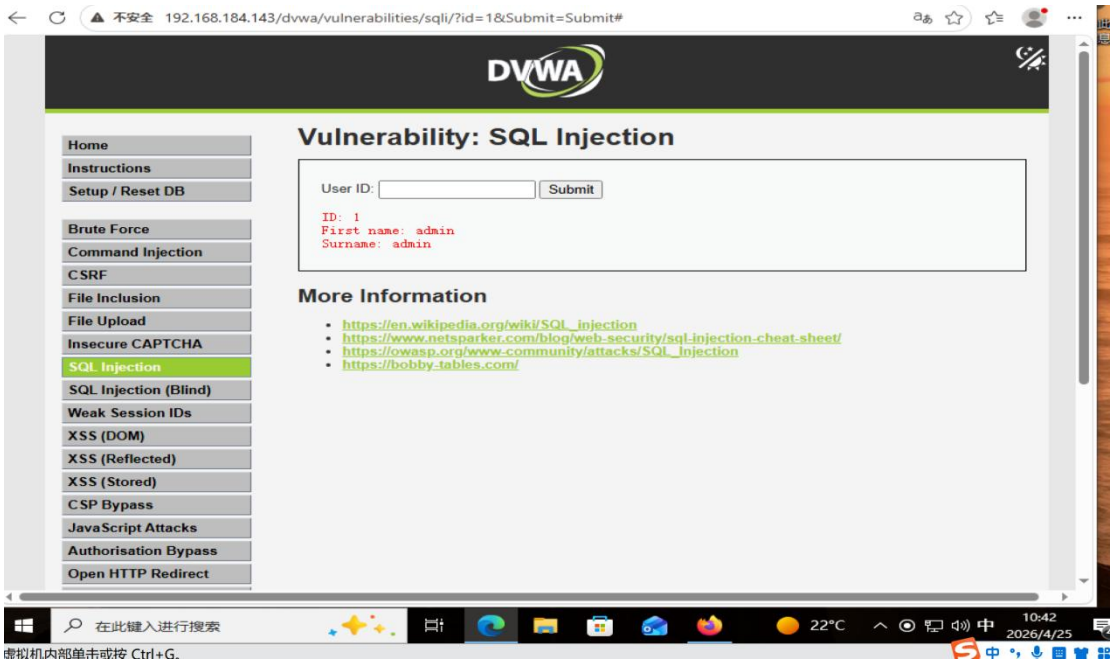
3. 返回结果的差异

数量差异：正常输入返回 1 条记录，异常输入返回 5 条记录（所有用户）。
内容差异：
正常输入：只能看到 admin 用户的信息。
异常输入：可以看到数据库中所有用户的名字，包括 Gordon Brown、Hack Me、Pablo Picasso、Bob Smith。
查询逻辑差异：
正常输入：后端 SQL 大致为 `SELECT first_name, surname FROM users WHERE user_id = 1;`，只查 ID=1 的用户。
异常输入：`1’or 1=1#` 使 SQL 变成 `SELECT ... WHERE user_id = ’1’or 1=1 # ...`，其中 `1=1` 恒为真，`#` 注释掉后续条件，导致返回所有用户。

4. 为什么这种差异不像“正常查询变化”

正常查询变化应该是：
输入不同的 ID（如 1、2、3）→ 返回不同用户，但**每次仍然只返回一条记录。
输入不存在的 ID → 返回空结果或“未找到”。
而这里的差异是：
输入 1’or 1=1# 后，返回结果从 **1 条变成了全部 5 条**。
这种“一次返回整个表”的行为，不符合正常业务逻辑。正常业务中，用户查询功能不应该有能力看到所有其他人的信息。
这个差异支撑的判断是：后端没有对输入进行过滤或转义，单引号没有被正确处

理，攻击者可以注入额外的 SQL 代码（or 1=1#）来改变查询的原始意图，说明存在 SQL 注入漏洞。



任务 3：机制分析与差异比较

1. 输入如何进入查询结构

用户输入的字符串通过 Web 应用的前端表单/URL 参数提交至后端服务器。后端代码（如 PHP、Python）直接将用户输入拼接到 SQL 查询语句的字符串中，未经过滤或参数化处理。拼接后的完整 SQL 语句被数据库解析，再执行，最终根据查询结果回显数据或产生可观测的行为差异。

链条：输入 → 拼接 → 解析 → 执行 → 回显/行为差异

2. 为什么根因是“结构被污染”而不是“字符本身危险”

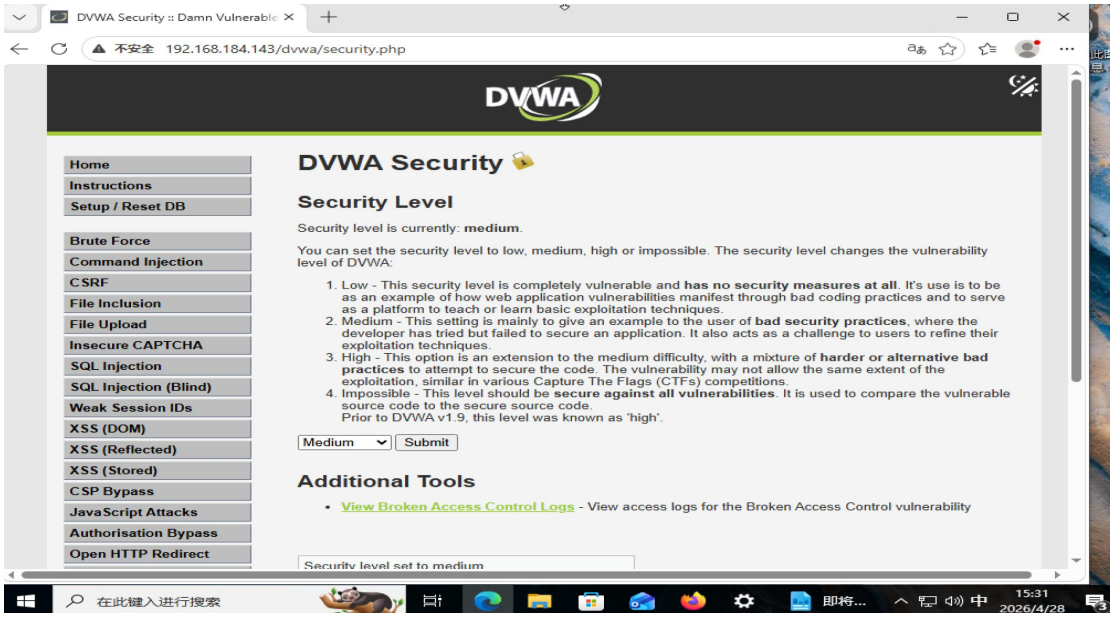
字符本身是中性的：单引号、分号等特殊字符在普通数据中是安全的，只有在特

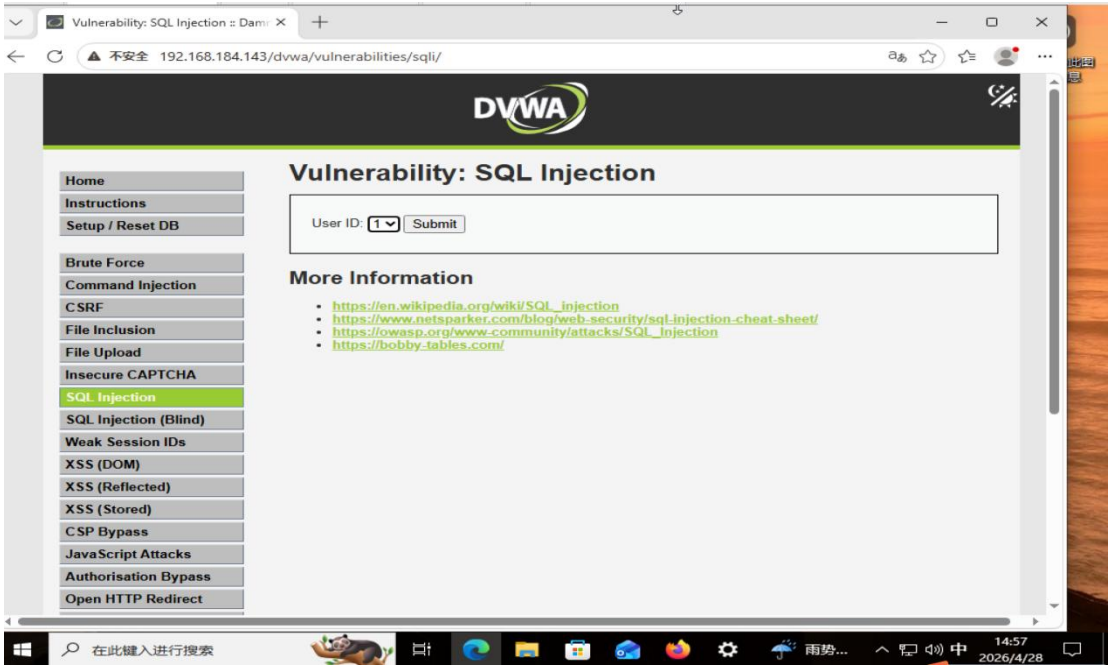
定上下文（SQL 语句结构）中被解释为代码分隔符时才会引发攻击。
根因是结构被污染：开发者将用户输入当作 SQL 代码的一部分进行拼接，而非当作纯数据。攻击者利用这一点注入额外的 SQL 片段，改变了原查询的逻辑结构（如恒真条件’ or 1=1 --使 WHERE 子句永远成立）。
结论：危险的不是字符，而是错误的拼接方式导致程序无法区分“代码”与“数据”，即结构被污染。

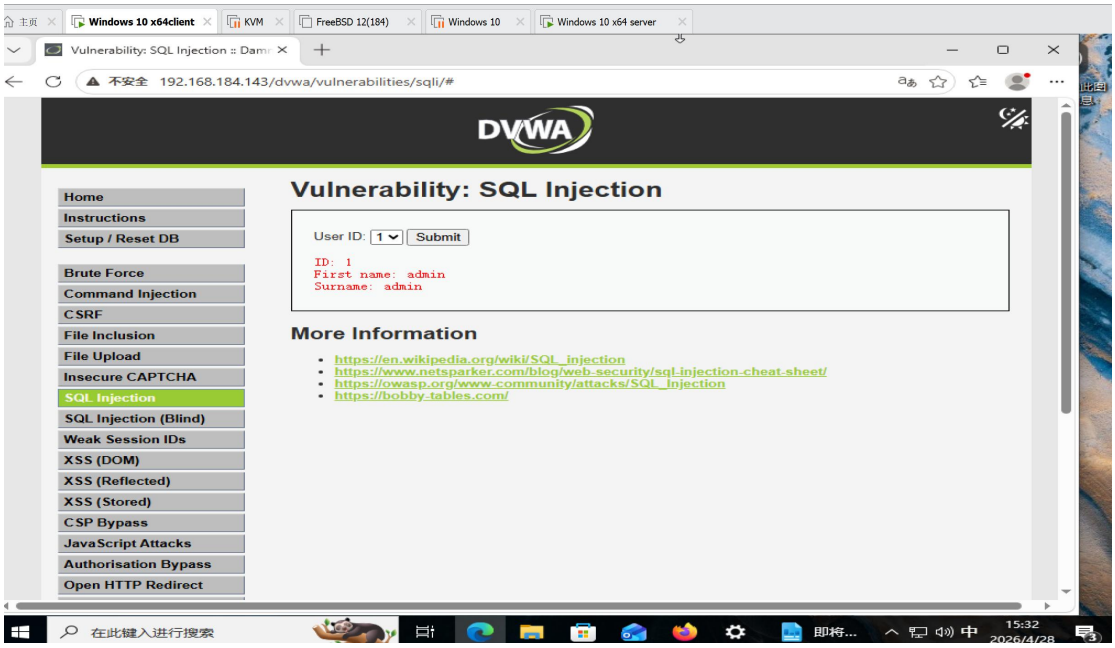
3. Low 与 Medium 的差异在哪里（以 DVWA 为例）
级别 防御方式 核心差异
Low 无任何过滤，直接拼接 可随意使用’ or 1=1 --、union select 等经典注入
Medium 使用 mysqli_real_escape_string()转义单引号等字符，且限制输入长度 转义后注入语句中的单引号被自动添加反斜杠（\’），导致无法提前闭合字段；同时长度限制阻止过长 payload
差异本质：Medium 引入了输入转义，但仍然使用拼接，攻击者可能通过其他方式绕过（如不依赖单引号的数字型注入、宽字节注入等）。但相比 Low，直接注入难度增加。

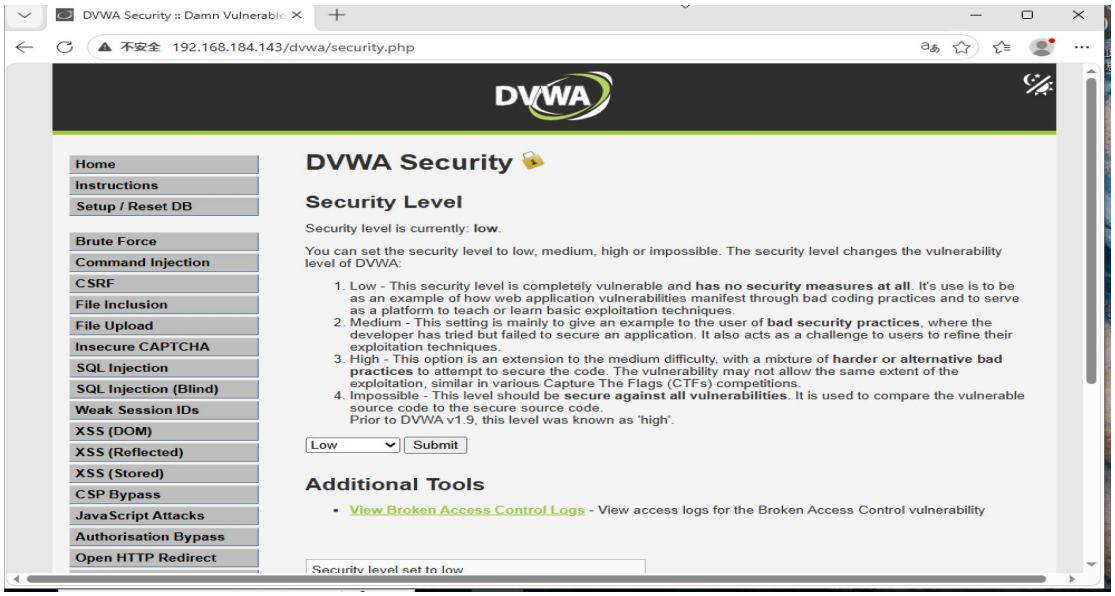
4. 普通 SQL 注入与 Blind SQL Injection（盲注）的差异在哪里
维度 普通 SQL 注入 盲注
回显信息 查询结果直接显示在页面上（如用户名、数据表内容） 不显示查询结果，页面只返回“成功/失败”或“正常/异常”
攻击手法 使用 union select 直接提取数据 通过布尔条件 (and 1=1 / and 1=2) 或时间延迟 (sleep()) 逐个字符推测数据
效率 高，可直接获取批量数据 低，需要大量请求与二分法猜测
依赖条件 页面有完整数据回显区域 只需页面存在可观测的差异（内容变化、响应时间、HTTP 状态码）

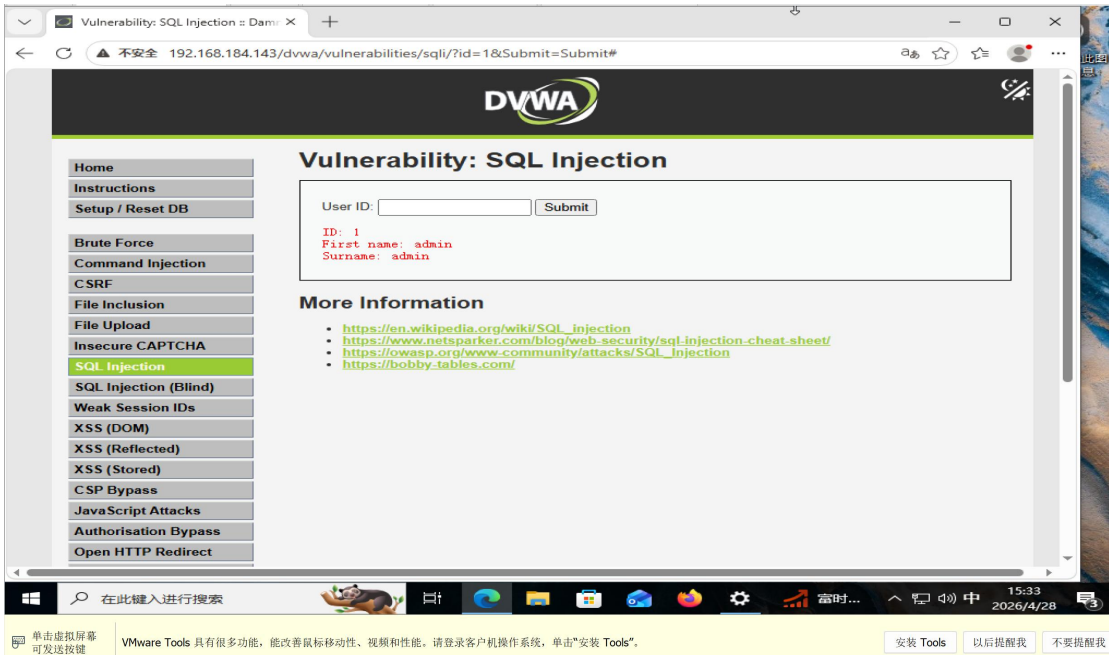
5. 为什么盲注和普通注入根因相同，但证据形式不同
根因相同：都是因为用户输入被拼接进 SQL 语句，污染了查询结构，导致数据库执行了攻击者期望的额外逻辑。
证据形式不同：差异来自 Web 应用的输出方式：
普通注入场景：应用会将数据库查询的结果直接回显给用户，攻击者能从页面上看到返回的数据。
盲注场景：应用只返回“查询成功（如跳转到正常页面）”或“查询失败（如报错或显示‘无数据’）”，甚至不改变页面内容，仅通过时间长短反映执行路径。攻击者必须通过构造若干真/假条件，观察页面行为的微小差异来推断数据。

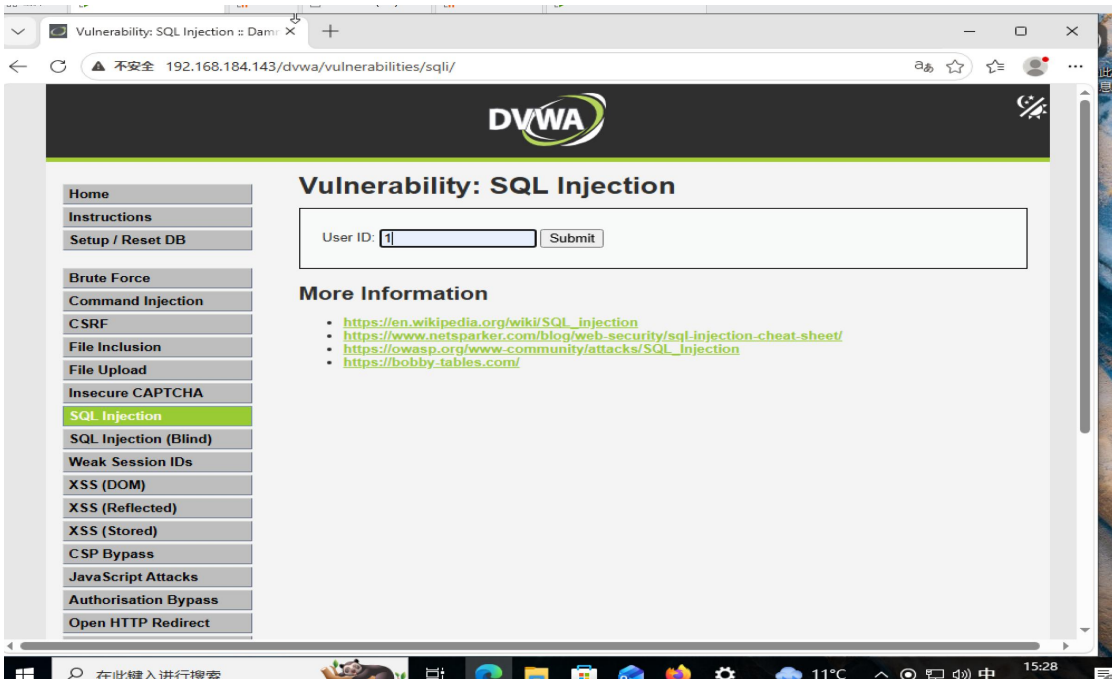

DVWA Security :: Damn Vulnerable
不安全 192.168.184.143/dvwa/security.php
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
DVWA Security
Security Level
Security level is currently: medium.
You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:
1. Low - This security level is completely vulnerable and has no security measures at all. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of bad security practices, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of harder or alternative bad practices to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be secure against all vulnerabilities. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.
Medium
Submit
Additional Tools
View Broken Access Control Logs - View access logs for the Broken Access Control vulnerability
Security level set to medium
在此键入进行搜索
即将...
中
15:31
2026/4/28

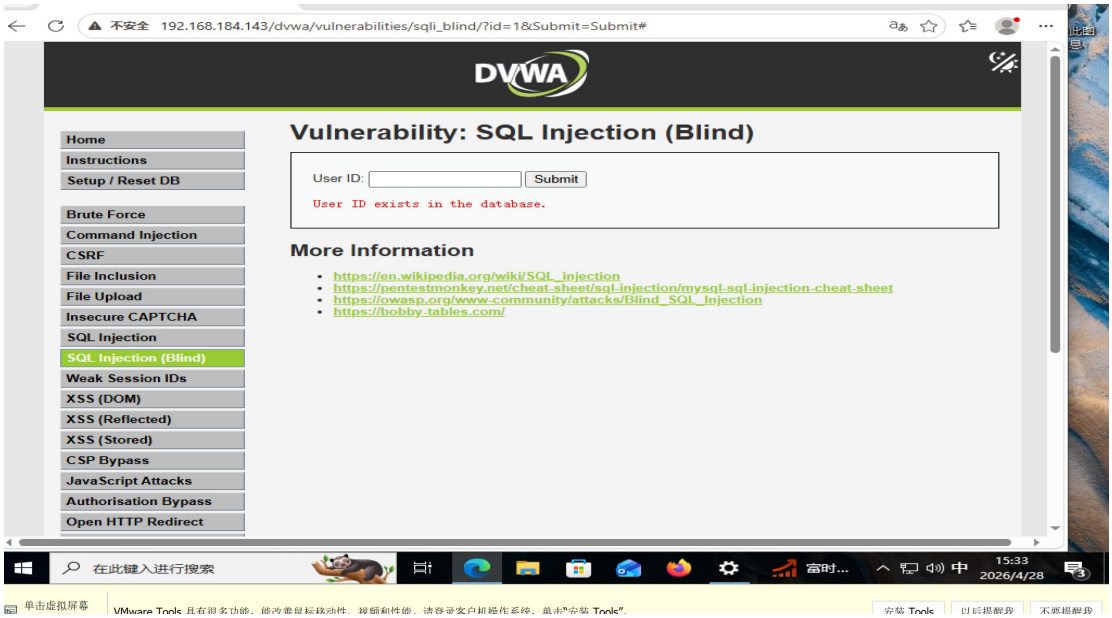

Vulnerability: SQL Injection :: Dam
不安全 192.168.184.143/dvwa/vulnerabilities/sqli/
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Vulnerability: SQL Injection
User ID: 1
Submit
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/
在此键入进行搜索
雨势...
中
14:57
2026/4/28

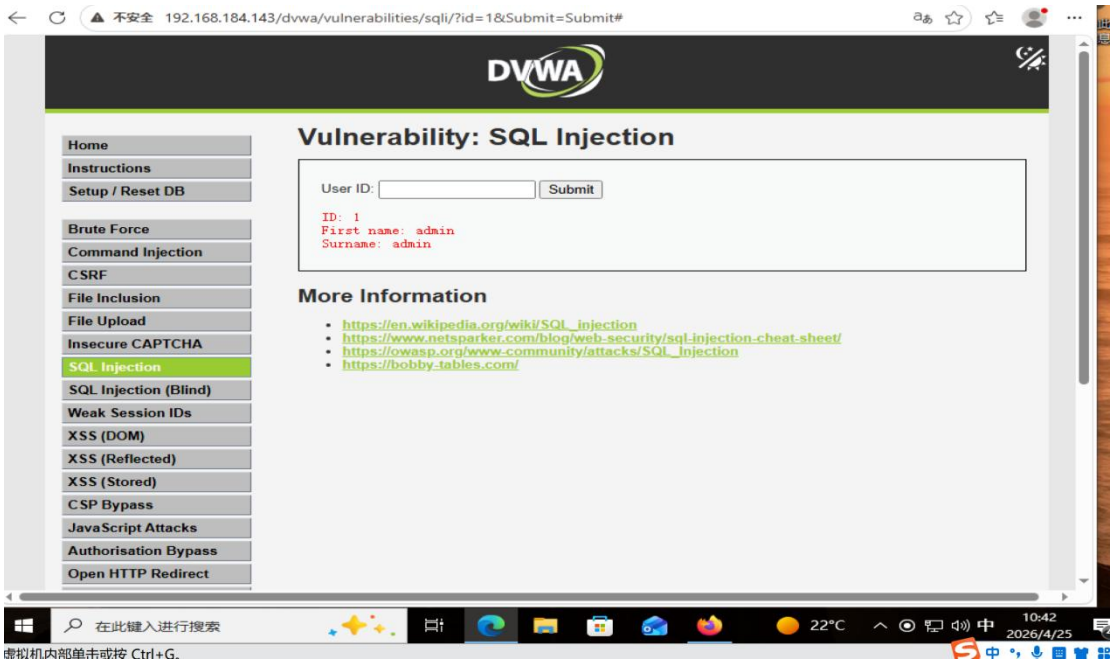

Windows 10 x64client
KVM
FreeBSD 12(184)
Windows 10
Windows 10 x64 server
Vulnerability: SQL Injection :: Dam
不安全 192.168.184.143/dvwa/vulnerabilities/sqli/#
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Vulnerability: SQL Injection
User ID: 1 Submit
ID: 1
First name: admin
Surname: admin
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/
在此键入进行搜索
即将...
15:32
2026/4/28


DVWA Security :: Damn Vulnerable
不安全 192.168.184.143/dvwa/security.php
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
DVWA Security
Security Level
Security level is currently: low.
You can set the security level to low, medium, high or impossible. The security level changes the vulnerability level of DVWA:
1. Low - This security level is completely vulnerable and has no security measures at all. It's use is to be as an example of how web application vulnerabilities manifest through bad coding practices and to serve as a platform to teach or learn basic exploitation techniques.
2. Medium - This setting is mainly to give an example to the user of bad security practices, where the developer has tried but failed to secure an application. It also acts as a challenge to users to refine their exploitation techniques.
3. High - This option is an extension to the medium difficulty, with a mixture of harder or alternative bad practices to attempt to secure the code. The vulnerability may not allow the same extent of the exploitation, similar in various Capture The Flags (CTFs) competitions.
4. Impossible - This level should be secure against all vulnerabilities. It is used to compare the vulnerable source code to the secure source code.
Prior to DVWA v1.9, this level was known as 'high'.
Low Submit
Additional Tools
View Broken Access Control Logs - View access logs for the Broken Access Control vulnerability
Security level set to low


Vulnerability: SQL Injection :: Dam
不安全 192.168.184.143/dvwa/vulnerabilities/sqli/?id=1&Submit=Submit#
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Vulnerability: SQL Injection
User ID:
Submit
ID: 1
First name: admin
Surname: admin
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/
在此键入进行搜索
15:33
2026/4/28
单击虚拟屏幕可发送按键
VMware Tools 具有很多功能，能改善鼠标移动性、视频和性能。请登录客户机操作系统，单击"安装 Tools"。
安装 Tools
以后提醒我
不要提醒我


Vulnerability: SQL Injection :: Dam
不安全 192.168.184.143/dvwa/vulnerabilities/sqli/
DVWA
Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass
JavaScript Attacks
Authorisation Bypass
Open HTTP Redirect
Vulnerability: SQL Injection
User ID: 1
Submit
More Information
https://en.wikipedia.org/wiki/SQL_injection
https://www.netsparker.com/blog/web-security/sql-injection-cheat-sheet/
https://owasp.org/www-community/attacks/SQL_Injection
https://bobby-tables.com/
11°C
15:28





任务 4：SQL 注入防护清单与解释

1. 参数化查询（Prepared Statements / Parameterized Queries）

解决什么问题

将 SQL 代码与数据分离，避免用户输入被解释为 SQL 语法结构，从根本上消除拼接 SQL 带来的注入可能。

控制哪一层

根因控制（注入漏洞产生的根本原因是数据与代码未分离）。

不能替代什么

不能替代输入校验（某些场景仍需校验业务合法性，如枚举值、长度），也不能替代权限管理（参数化查询不防止越权查询）。

2. 输入校验（Input Validation）
解决什么问题
在数据进入 SQL 语句之前，拒绝明显恶意或不符合格式的输入（如 ID 字段只允许数字、日期字段校验格式），减少攻击面并作为第二道防线。
控制哪一层
暴露面控制（缩小可被攻击的输入范围）。
不能替代什么
不能替代参数化查询（仅靠白名单校验无法覆盖所有动态场景，如 LIKE、IN 子句仍可能被绕过）。

3. 错误处理（Error Handling）
解决什么问题
防止数据库错误信息（如语法错误、列名暴露、堆栈跟踪）直接返回给客户端，避免攻击者通过错误回显进行信息收集和注入试探。
控制哪一层
暴露面控制（减少攻击者可获得的信息反馈通道）。
不能替代什么
不能替代参数化查询（错误隐藏不修复注入漏洞本身，只是让利用变难）。

4. 最小权限（Least Privilege）
解决什么问题
数据库账户仅授予应用程序必要的权限（例如只读账户不授予 INSERT/DROP/ALTER），限制即使发生注入，攻击者能造成的破坏范围。
控制哪一层
后果范围控制（压缩漏洞可利用后的影响半径）。
不能替代什么
不能替代参数化查询（最小权限不影响注入是否发生，只影响后果）。

5. 使用存储过程并限制动态 SQL（Stored Procedures without Dynamic SQL）
解决什么问题
将 SQL 逻辑封装在数据库端，并避免在存储过程内部使用字符串拼接/EXEC 或 `sp_executesql`动态执行用户输入，减少应用层直接拼接 SQL 的机会。
控制哪一层
暴露面 + 根因辅助（减少 SQL 构造点；若存储过程内无动态 SQL 则从根本上杜绝该存储过程的注入）。
不能替代什么
不能替代参数化查询原则（错误的存储过程写法仍会引入注入），也不能替代输入校验和权限控制。

收口项

收口项 1：SQL 注入因果链/防护关系图

因果环节	说明	对应防护措施（控制层）
1. 输入点	用户可通过表单、URL 参数、Cookie、HTTP 头等向应用传递不可信数据	输入校验（暴露面控制）
2. 查询构造	应用程序使用字符串拼接、格式化等方式将用户输入直接嵌入 SQL 语句	参数化查询（根因控制）存储过程（无动态 SQL）（暴露面 / 根因辅助）
3. 数据库解析 / 执行	数据库引擎无法区分代码与数据，将恶意输入当作 SQL 语法执行	（无直接防护，依赖前两步）
4. 结果差异或行为证据	攻击者通过报错、页面内容变化、时间延迟、布尔差异等推断注入是否成功	错误处理（暴露面控制）（隐藏证据，减慢攻击节奏）
5. 根因判断	注入的根本原因是：SQL 代码与数据未分离，而非输入本身有恶意	参数化查询（根因控制）
6. 后果范围	注入成功后，攻击者可能执行越权查询、删表、写文件、提权等	最小权限（后果范围控制）

收口项 2：学生自评表

我本章最清楚的一点是什么？

“参数化查询控制的是根因”与“最小权限控制的是后果范围”之间的区别。我理解了 SQL 注入的本质是代码/数据未分离，不是输入本身有问题。

我最容易混淆的一点是什么？

输入校验到底算根因还是暴露面。

→ 之前会误以为校验能根本解决注入，后来明白它只是缩小攻击入口，不能替代参数化查询。

我本次作业中哪一段最需要教师复核？

“存储过程是否属于根因控制”——我判断是暴露面/根因辅助，因为如果存储过程内部仍然使用动态 SQL（EXEC、拼接），同样会存在注入风险。希望老师确认这个分层是否正确。

如果重做一次，我会优先改哪一部分？

会把防护清单中每条措施的“不能替代什么”写得更具体，例如明确写出“参数化查询不能替代最小权限，因为参数化不防止越权”。

收口项 3：AI 输出审核与人工复核记录

1. 哪些内容使用过 AI 辅助

任务 4 防护清单、收口项 1 关系表：使用 AI 生成初版结构
收口项 2 自评表：未使用 Ai

3. 自己复核后保留的判断
参数化查询→根因控制（OWASP 依据）
输入校验→暴露面控制（校验可绕过，不解决根因）
最小权限→后果范围控制（不阻止漏洞，只限制影响）

3.AI 输出不准确并修正的内容

AI 输出（错误）	修正后
输入校验属于根因控制	属于暴露面控制
存储过程可完全替代参数化查询	存储过程内若用动态 SQL 仍可注入
错误处理属于后果范围控制	属于暴露面控制（隐藏证据）

4. 如何确认结论与证据一一对应
采用反向验证法：对每条结论提问“如果只有这条措施，漏洞/后果是否还存在”，逐条验证分层是否正确。